

COS 214 Practical 3

EventFlow: Designing a Live Event Coordination Engine

- Date Released: **18 August 2026**
 - Date Due: **1 September 2026** at 23:59
 - Team Size: **3 students**
 - Language: **C++11**
 - Core topics: **Observer, Composite, UML sequence diagrams, Doxygen, Git/GitHub**
 - Marks: **300**
-

1 Introduction

A successful live event is not one object doing everything. A festival, conference, sports day, open day, exhibition or community event is made up of many areas and operational units that must coordinate while the event changes around them. A venue may contain zones; zones may contain stages, gates, vendors, staff teams and sub-zones; and a central control point may need to inform many interested parts of the event when something changes.

In this practical you will design and implement **EventFlow**, a reusable event-coordination engine. Unlike Practical 2, you are not given a broken implementation to repair. You are given a scenario, a deliberately incomplete UML design, a set of required interactions and many behavioural stories. You must complete the design, make it your own, implement it in C++11 and prove how objects collaborate using UML sequence diagrams.

The event itself is yours to invent. Your team may build a music festival, gaming convention, university open day, food market, sports tournament, technology expo, book fair, film festival, charity event, motorsport weekend, or another realistic event approved by your tutor. The architecture must support the required behaviours regardless of theme.

Central idea. Composite answers: *“what is contained inside this part of the event?”* Observer answers: *“who needs to hear about this change?”* A class may participate in both collaborations without the two patterns becoming the same pattern.

2 Learning Outcomes

After completing this practical you should be able to:

- apply the Gang of Four **Observer** pattern as a genuine one-to-many dependency;
- register and deregister observers safely and notify only currently registered observers;
- reason about a **push** versus **pull** approach to Observer state transfer;
- apply the Gang of Four **Composite** pattern to a genuine part-whole tree;
- distinguish ownership from observation and from ordinary association;
- make one class participate in more than one pattern collaboration for different reasons;
- model detailed object interactions with UML **sequence diagrams**;
- use sequence frames, lifelines, activation, calls, returns, self-messages, creation/destruction and combined fragments such as **loop**, **alt** and **opt**;
- maintain correct C++11 polymorphic destruction and explicit ownership;
- document a C++ design using **Doxygen**; and
- collaborate professionally through **Git and GitHub** using meaningful commits and shared repository history; feature branches and pull requests are recommended as good practice.

3 Non-negotiable Rules

1. Complete this practical in a **team of three**. All three members must understand the complete submission.
2. Compile using `-std=c++11`. Your project must be built using a **Makefile**. The marker will build your submission by running `make`; no alternative build system is required or assessed. The Makefile must produce an executable named `eventflow`.
3. Every polymorphic base class must have a **virtual** destructor.
4. Your code will be checked for memory leaks. Every owning relationship must have a clear destruction policy.

5. You may not replace Observer or Composite with type checks, giant **switch** statements, large **if/else** dispatch chains, or manual loops in **main.cpp** that know every concrete class.
6. Your diagrams and code must agree. A message shown in a sequence diagram must correspond to a real operation in your final design unless the diagram is explicitly labelled as a design proposal.
7. Public classes and public operations must be documented with Doxygen comments.
8. Your event must contain at least **three levels of Composite nesting** below the client/root boundary and at least **five distinct concrete leaf types** across the team design.
9. Your final running system must demonstrate observers being attached, detached and notified at runtime.

GITHUB IS COMPULSORY. If your team does not use GitHub for the development of this practical, 25% will be deducted from the final practical mark. A repository uploaded only at the end does not count as meaningful GitHub use. The repository history must show development over time by all three team members.

4 Submission and Demonstration

Submit one archive containing the source code and one PDF containing the required design material. Your GitHub repository must remain accessible to the teaching team until marking is complete.

Your repository must contain at least:

- all **.h** and **.cpp** files;
- **main.cpp**;
- **Makefile**;
- **README.md**;
- **Doxyfile**;
- a **docs/** directory containing the submitted diagrams/report or exported copies; and
- evidence of ongoing GitHub development in the repository history, with meaningful contributions from all three team members.

At the demonstration, the tutor may ask *any* team member to locate code corresponding to a sequence-diagram message, explain an ownership decision, explain why an observer receives or does not receive a notification, or explain part of the team's Git history.

5 Mark Allocation

Task	Marks
1. Event concept and completed architecture	45
2. Composite event structure	35
3. Observer notification system	40
4. Event rules and dynamic behaviour	25
5. Sequence diagram portfolio	100
6. Doxygen documentation	20
7. Git and GitHub engineering workflow	25
8. Integration and demonstration	10
TOTAL	300

6 Scenario: EventFlow

Your team is developing software for a live event organiser. The organiser runs events made up of nested operational areas. A root event may contain zones; a zone may contain sub-zones and individual operational units; and all of these should be treated uniformly by parts of the application that merely want to open, close, report status or calculate capacity.

A possible event structure is shown below. **This is an example only; do not copy it unchanged.**

```

City Lights Festival
|
+-- Main Stage Zone
| +-- Main Stage
| +-- Front Gate
| +-- Medical Team
| +-- Food Court
| +-- Taco Vendor
| +-- Coffee Vendor
|
+-- River Zone
| +-- River Stage
| +-- Security Team
| +-- Shuttle Stop
|
+-- Family Zone
    +-- Workshop Tent
    +-- Information Desk
    +-- First Aid Point

```

Individual units react differently. For example, a **Stage** may pause a performance, a **Gate** may stop admitting attendees, a **Vendor** may suspend service, and a **MedicalTeam** may remain operational during the same event-wide alert. Your design must achieve different behaviour through polymorphism and object collaboration, not explicit type inspection.

At the same time, the event contains a notification structure. A central control centre can issue changes such as:

- opening or closing areas;
- schedule changes;
- capacity warnings;
- weather alerts;
- transport delays;
- evacuation or temporary pause instructions;
- resumption notices; and
- custom event-specific notices invented by your team.

Some event areas receive notifications from above and then notify interested units below. Consequently, an event-area class may need to be both an **Observer** and a **Subject**. That is not a mistake: the two roles describe different collaborations.

7 Required Conceptual Model

The design must contain a common event component abstraction. You may choose your own names, but for this specification the following generic names are used:

- **EventComponent**: common Component interface;
- **EventUnit**: a Leaf such as a stage, vendor, gate, information desk, shuttle stop or staff team;
- **EventGroup**: a Composite such as a zone, venue, hall, precinct or programme area;
- **Subject**: abstraction for something that can be observed;
- **Observer**: abstraction for something that receives notifications;
- **EventControl**: an event-level coordinator or concrete subject; and
- an order/notice representation of your design.

Your team may rename all domain classes. The GoF roles, however, must remain identifiable.

7.1 Minimum common event operations

At minimum the common event abstraction must support sensible versions of:

```

virtual void open() = 0;
virtual void close() = 0;
virtual void reportStatus() const = 0;
virtual int getCapacity() const = 0;

```

You may adapt signatures and add operations, but the client must be able to invoke useful behaviour through the base Component interface without knowing whether it holds one Leaf or an entire Composite.

7.2 Minimum Observer operations

Your Observer collaboration must contain operations equivalent in intent to:

```
attach(Observer* observer)
detach(Observer* observer)
notify()
update(...)
```

You must decide whether the subject owns its observers. Your answer must be consistent with your destructor logic and UML diamonds/multiplicities.

7.3 Push or pull

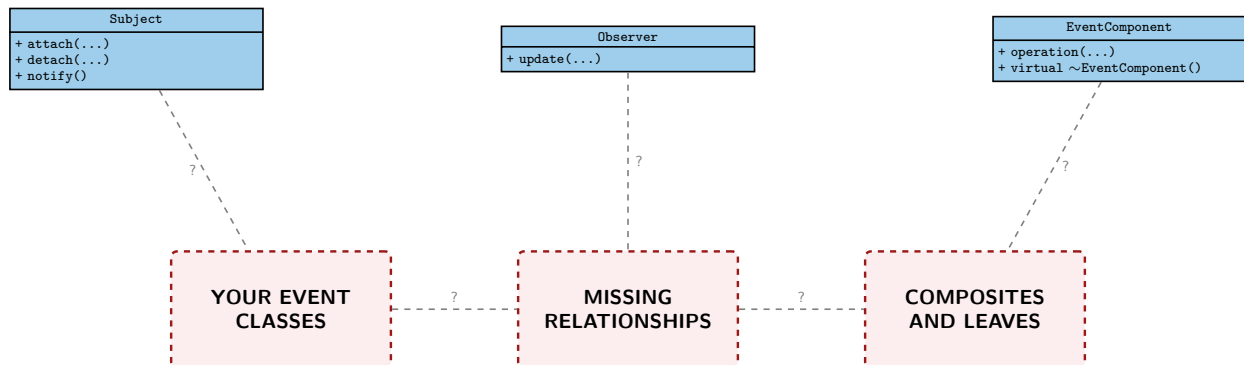
Your team must choose one of the following and use it consistently:

- **Push:** the subject sends relevant event information to `update(...)`; or
- **Pull:** `update()` tells the observer that something changed and the observer queries the subject for state.

You will defend this decision in the PDF and model it explicitly in at least one sequence diagram.

8 Figure 1: The Incomplete Architecture

Figure 1 is deliberately only a **starting point**. It shows the abstractions that must exist somewhere in your design, but it does not tell you how they connect. You must decide which concrete classes are needed, where the two patterns meet, what is owned, what is merely observed, and which relationships and multiplicities belong in the completed UML.



The diagram is intentionally incomplete. Do not assume that one missing box equals one missing class. Your final UML may introduce the concrete Subject, concrete Observers, Composite, Leaves and other event-specific classes that your scenario needs. You are expected to complete the design rather than reproduce a supplied layout.

Task 1: Event concept and completed architecture (45 marks)

Your team must first decide what kind of event EventFlow is coordinating. The event must be large enough to justify nested structure and changing notifications.

- 1.1 Define your event in approximately one page or less. Give it a name, explain what happens there, and identify at least **five concrete kinds of event units** and at least **three kinds of grouping/area** that could appear in the Composite tree.
- 1.2 Identify the GoF participants of both Observer and Composite in your design. If one class participates in more than one role, list each role separately and explain the collaboration in which that role applies.
- 1.3 Complete the UML class diagram for your full EventFlow design. Show abstract classes/operations, inheritance, associations, multiplicities, ownership, observer registrations, relevant attributes and operations, and a virtual destructor on every polymorphic base. Your diagram must include at least five concrete leaf types and enough structure to support every scenario in Task 5.
- 1.4 Write a design rationale. Explain: (a) why your Composite is a genuine part-whole tree; (b) why Observer is required rather than direct hard-coded calls; (c) who owns event components; (d) who owns or does not own observer references; and (e) why any class that is both Subject and Observer is not a misuse of either pattern.

Task 2: Composite event structure (35 marks)

Implement the structural side of EventFlow using Composite.

- 2.1 Implement the abstract Component and at least **five** concrete Leaf types appropriate to your event. Concrete leaves must have meaningfully different output/behaviour; changing only a printed name is not sufficient.
- 2.2 Implement your Composite class. It must contain children that are Components, support at least **add(...)** and a sensible removal operation, and recursively implement the required common operations. A Composite must be able to contain both Leaves and other Composites.
- 2.3 Build a sample event with at least **three levels of nesting**. Draw an **object diagram** showing the concrete object instances and the ownership tree.
- 2.4 Demonstrate destruction of the root and explain why the complete owned subtree is released exactly once. If an object can be transferred between composites, define the ownership rules for that transfer clearly.

Task 3: Observer notification system (40 marks)

Implement runtime event communication using Observer.

- 3.1 Implement the Subject abstraction and its registration list. **attach** and **detach** must behave sensibly for duplicate registration and attempts to detach an observer that is not registered. State your policy.
- 3.2 Implement the Observer abstraction and the required concrete observing behaviour. Destruction must not leave dangling registrations. Explain whether observers detach themselves, subjects clean registrations, or your design guarantees lifetime ordering in another defensible way.

- 3.3 Implement at least **six notice/order types**, including at least one ordinary operational change, one capacity-related change and one safety-related change. Examples are `OPEN`, `CLOSE`, `SCHEDULE_CHANGE`, `CAPACITY_ALERT`, `WEATHER_ALERT`, `PAUSE`, `RESUME`, `EVACUATE`.
- 3.4 Make at least one Composite-level class capable of receiving a notification from above and then notifying interested observers below. Demonstrate a notification cascading through at least three runtime levels.
- 3.5 Choose push or pull for your Observer implementation. Explain the trade-off and show the exact state/message information transferred in your design.

Task 4: Event rules and dynamic behaviour (25 marks)

Your event should feel like a system rather than a demonstration of isolated pattern classes.

- 4.1 Define at least **five event rules** that cause concrete units to react differently to the same notice. Example: during a weather alert an outdoor stage pauses, an indoor hall remains open, a shuttle changes route and a medical unit remains active. Achieve these differences through polymorphism and object state, not type checking in a controller.
- 4.2 Implement at least one runtime reorganisation. For example, transfer a cleaning team, vendor, shuttle or information unit from one area to another. The Composite ownership relationship and Observer registrations must both be updated correctly.
- 4.3 Implement at least one condition-based decision that will later appear inside an `alt` or `opt` combined fragment, such as “if capacity $\geq$ threshold”, “if outdoor”, “if service is available” or an event-specific rule.
- 4.4 Add at least **three original features** that are not explicitly required above. They must fit your event and should create interesting interactions worth modelling. Briefly explain why each feature could be added without turning one class into a god object.

Task 5: Sequence diagram portfolio (100 marks)

Sequence diagrams are the heart of this practical. Submit **four substantial sequence diagrams**. Each diagram must be a UML sequence diagram, not a flowchart. Use object-instance lifelines (for example `mainZone : EventGroup`), real method signatures from your implementation, activation where useful, dashed returns for non-void calls, and correct creation/destruction notation when object lifetime is part of the scenario.

These four diagrams are intentionally broader than small one-operation diagrams. Each should tell a complete interaction story. Across the portfolio you must use `loop`, `alt` and `opt` at least once each. At least **three** diagrams must contain a UML combined fragment, and at least three diagrams must show nested calls through three or more collaborating objects.

- 5.1 **SD1 – Building and registering part of the event.** Show the client creating a root Composite, at least one nested Composite and at least two Leaves, adding them to the ownership tree, and establishing the Observer registrations needed for later notifications. Creation messages must begin lifelines at the correct time. The interaction must make it possible for a marker to distinguish *Composite ownership* from *Observer registration*.
- 5.2 **SD2 – Cascading event notification.** Show a realistic notice originating from an appropriate Subject and cascading through at least three runtime levels to multiple observers. Use a `loop` fragment where observers are notified. If you chose pull, show the observer querying the subject; if you chose push, show the information passed to `update(...)`. At least two concrete event units must react meaningfully to the notice.
- 5.3 **SD3 – Conditional event response and Composite behaviour.** Model a realistic situation such as a capacity threshold, weather warning, schedule disruption, temporary closure or service interruption. The interaction must include a meaningful Composite operation and use an `alt` fragment with guards. Include an `opt` fragment where it naturally fits. Show different concrete objects responding through polymorphism rather than explicit type checking.
- 5.4 **SD4 – Your signature event scenario.** Invent a substantial interaction that makes your team’s event recognisably different from another team’s system. It must involve at least six lifelines, at least two Composite levels, at least one Observer notification, and at least one change during execution, such as an observer joining/leaving, a unit being reassigned, a pause followed by resume, a safety response followed by recovery, or end-of-event shutdown. If ownership or registration changes, show the relevant detach/attach or transfer messages clearly. Use at least one combined fragment and ensure the scenario can be traced to your actual C++ implementation.

Sequence-diagram rule: sequence diagrams are not decorative documentation added at the end. They are part of the design. If a diagram exposes that your objects cannot perform the interaction cleanly, revise the class design and code.

Task 6: Doxygen documentation (20 marks)

Your source code must be documented with Doxygen comments and your repository must contain a working `Doxyfile`.

- 6.1 Add class-level Doxygen documentation to every class, explaining its responsibility and, where relevant, its GoF participant role.
- 6.2 Document every public operation. Use `@param` for parameters, `@return` for non-void results, and explicitly document ownership/lifetime expectations when raw pointers cross an interface.
- 6.3 Document at least three non-obvious design decisions using appropriate detailed comments, for example why observer pointers are non-owning or how a transfer between composites changes ownership.
- 6.4 Configure Doxygen so that running `doxygen Doxyfile` generates browsable documentation. Major warnings caused by undocumented public interfaces will lose marks.

A suitable style is:

```
/**
 * @brief Registers an observer for future event notifications.
 *
 * This Subject stores a non-owning pointer. The observer must remain
 * alive while registered and must be detached before destruction.
 *
 * @param observer Observer to register. Must not be nullptr.
 */
virtual void attach(Observer* observer) = 0;
```

Task 7: Git and GitHub engineering workflow (25 marks)

GitHub is part of the practical, not merely the place where the final code happens to live. You have only recently been introduced to branching workflows, so the emphasis in this practical is on **using GitHub consistently as a team** rather than mastering an advanced workflow.

- 7.1 Create one private GitHub repository for the team. The `README.md` must contain the event concept, team members, instructions for building with `make` and running the executable, Doxygen instructions and a brief architecture overview.
- 7.2 Use meaningful commits throughout development. The repository history must show genuine development over time and meaningful contributions from **all three team members**. Commits such as `stuff`, `final`, `changes`, `final2` or a single end-of-practical dump will receive little or no credit.
- 7.3 Use GitHub to coordinate the team. Feature branches and pull requests are **recommended** and you are encouraged to practise them, but there is no required number of branches or pull requests for this practical. You may also demonstrate collaboration through issues, commit history and clear division of work. You will not lose marks simply because your team used a simpler branching workflow, provided the repository clearly shows ongoing collaborative development.
- 7.4 In your submitted PDF, include a short GitHub workflow reflection (approximately half a page). Explain how the three members divided the work, how changes were integrated, and identify two or three commits or other repository activities that show meaningful collaboration. If your team used branches or pull requests, briefly explain how they helped.

No GitHub development history = 25% deduction from the final practical mark, in addition to marks lost in this task. GitHub used only as a last-minute upload location will be treated as non-compliance.

Task 8: Integration and demonstration (10 marks)

- 8.1 Your `main.cpp` must build and run a coherent event simulation that demonstrates: construction of the Composite; observer registration; at least three different notices; at least one cascading notification; a registration change; a Composite traversal/query; a runtime reorganisation; and clean shutdown.
- 8.2 Prepare a demonstration route of approximately five minutes. Any team member must be able to drive the program and answer questions about the code, diagrams, ownership and Git history.

9 Sequence Diagram Quality Checklist

Before submitting, compare every sequence diagram against this checklist:

- Does every lifeline represent an **object instance**, not merely a class name?
- Does creation occur where the object actually comes into existence?
- If an object is destroyed, does its lifeline end at the destruction point?
- Are method-call labels signatures rather than snippets of C++ syntax such as `ptr->method()`?

- Are return values shown with dashed arrows where a return matters?
- Are self-calls drawn as self-messages?
- Is iteration shown with a `loop` frame rather than writing the same arrow five times?
- Are alternatives shown with guards such as `[capacity >= limit]` and `[else]`?
- Does the ordering of messages match the code?
- Can you point from every important message to the corresponding operation in the class diagram?
- Is the diagram readable when placed in the submitted PDF at normal zoom?

10 Creativity Requirements

Two teams may both choose festivals and still build very different systems. Your design must have a recognisable identity. The following are examples of directions you might explore; they are not requirements and copying the whole list earns no creativity credit:

- a conference where rooms observe programme changes and displays observe rooms;
- a sports tournament with venues, courts, officials, scoreboards and spectator gates;
- a food festival where precincts contain vendors and services, with allergen or stock alerts;
- a university open day with faculties, buildings, lecture venues, shuttle stops and information desks;
- a gaming convention with halls, tournaments, demo stations and queue-management units;
- a film festival with cinemas, screening rooms, ticket desks and schedule notifications;
- a community race with course sectors, aid stations, timing points and transport teams;
- a technology expo with halls, booths, presentation stages and attendee-capacity alerts.

Your event-specific classes should create genuinely different behaviour. A design with `LeafA`, `LeafB`, `LeafC` that merely print different strings is technically easy but conceptually weak.

11 Design Questions You Must Resolve

The specification intentionally leaves these decisions open:

- Is `EventControl` itself part of the Composite or only a Subject/client collaborator?
- Does every Composite observe its parent, or can some areas be structurally contained without observing notifications from that parent?
- Do all Leaves implement Observer, or only those interested in notifications?
- If a Composite owns a child, does it automatically register that child as an observer? If yes, justify the coupling; if no, show how the two relationships are managed separately.
- Can the same observer register with more than one subject?
- What happens if `attach()` is called twice with the same pointer?
- What happens when `detach()` cannot find the requested observer?
- What exactly is stored as “current notice” or subject state?
- Who initiates cascading notification after an observing Composite receives `update()`?
- What is the order of notifications, and does that order matter in your event?
- What happens if an observer changes registration while notification is in progress? You do not need multithreading, but your single-threaded policy must be sensible.
- Can an operational unit move between Composite parents? Who owns it during the transfer?
- How do you prevent double deletion if a unit is transferred?
- What does aggregate capacity/strength/status mean for your particular event?

You do not need to choose the same answers as another team. You do need a coherent answer that is reflected in UML, C++ and sequence diagrams.

12 PDF Submission Order

Your single submitted PDF must contain the following in this order:

1. event name, event description, team member names and student numbers;

2. completed UML class diagram;
3. Observer and Composite participant mapping;
4. design rationale, including ownership and push/pull choice;
5. Composite object diagram;
6. event rules and original features;
7. **SD1** Building and registering part of the event;
8. **SD2** Cascading event notification;
9. **SD3** Conditional event response and Composite behaviour;
10. **SD4** Signature event scenario;
11. Doxygen evidence: generated landing page or short screenshot/export showing successful generation;
12. GitHub repository link and the short workflow reflection required in Task 7;
13. a one-page team contribution statement identifying each member's main contributions and how the team coordinated its work; and
14. any written answers explicitly requested in the tasks above.

13 Final Advice

Do not begin by writing twenty classes. Begin with objects and interactions. Sketch the event tree. Decide who owns whom. Decide who needs to hear which changes. Draw two or three difficult sequence diagrams early. If those interactions are awkward, improve the design before committing to the implementation.

The strongest submissions will not be the ones with the most classes. They will be the ones in which the class diagram, ownership policy, Observer collaboration, Composite tree, C++ implementation and four sequence diagrams all tell the **same story**.